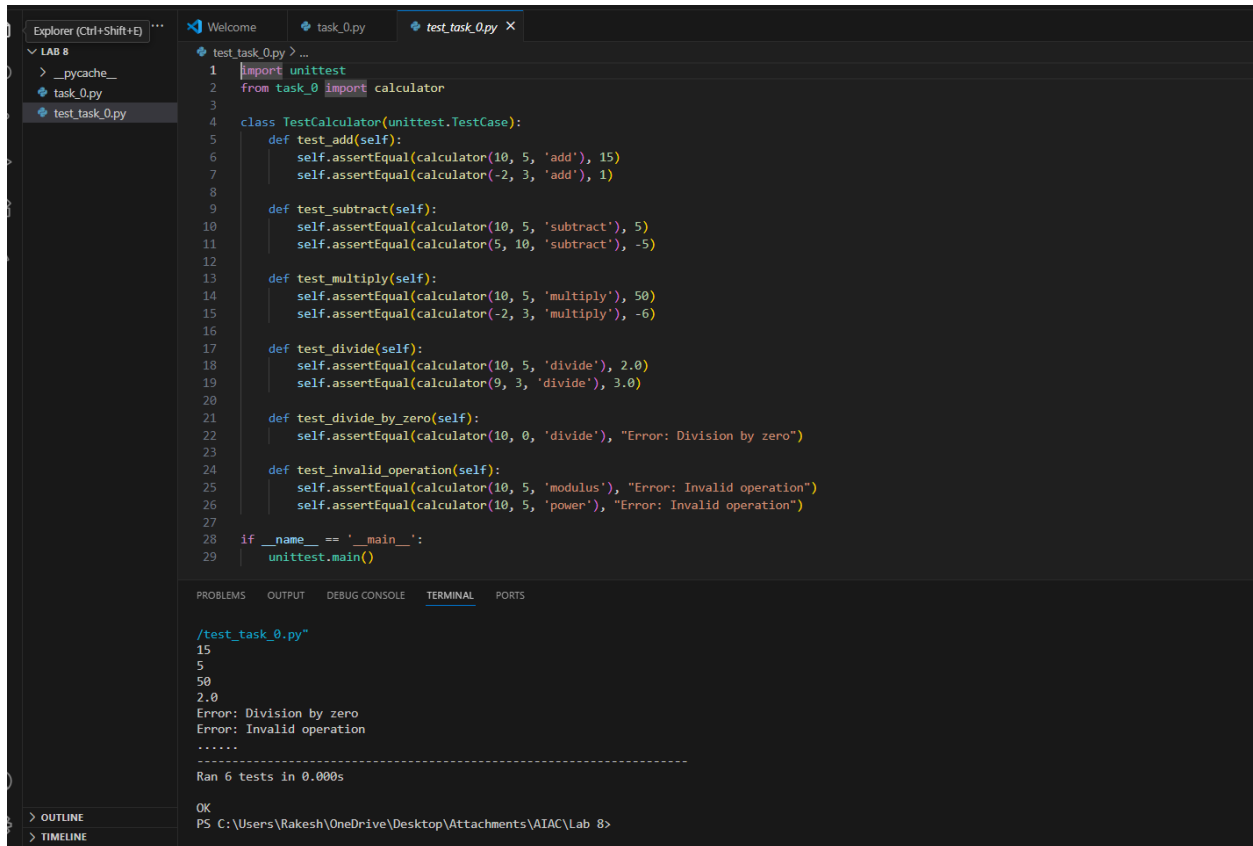


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear:2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week4 - Wednesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:8.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases Lab Objectives: - To introduce students to test-driven development (TDD) using AI code generation tools. - To enable the generation of test cases before writing code implementations.		Week4 - Wednesday

	• To reinforce the importance of testing, validation, and error handling. • To encourage writing clean and reliable code based on AI-generated test expectations. Lab Outcomes (LOs): After completing this lab, students will be able to: • Use AI tools to write test cases for Python functions and classes. • Implement functions based on test cases in a test-first development style. • Use unittest or pytest to validate code correctness. • Analyze the completeness and coverage of AI-generated tests. • Compare AI-generated and manually written test cases for quality and logic Task Description#1 Use AI to generate test cases for is_valid_email(email) and then implement the validator function. Requirements: • Must contain @ and . characters. • Must not start or end with special characters. • Should not allow multiple @. Expected Output#1 • Email validation logic passing all test cases Task Description#2 (Loops) • Ask AI to generate test cases for assign_grade(score) function. Handle boundary and invalid inputs. Requirements • AI should generate test cases for assign_grade(score) where: 90-100: A, 80-89: B, 70-79: C, 60-69: D, <60: F • Include boundary values and invalid inputs (e.g., -5, 105, "eighty"). Expected Output#2 Grade assignment function passing test suite Task Description#3 • Generate test cases using AI for is_sentence_palindrome(sentence). Ignore case, punctuation, and spaces Requirement • Ask AI to create test cases for is_sentence_palindrome(sentence) (ignores case, spaces, and punctuation). • Example: "A man a plan a canal Panama" → True Expected Output#3 • Function returns True/False for cleaned sentences • Implement the function to pass AI-generated tests. Task Description#4 • Let AI fix it Prompt AI to generate test cases for a ShoppingCart class (add_item, remove_item, total_cost). Methods: Add_item(name,orice) Remove_item(name)	
--	---	--

TEST CASE:



The screenshot shows a VS Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'LAB 8' with files 'task_0.py' and 'test_task_0.py'. The code editor shows the content of 'test_task_0.py', which is a Python test file using the unittest module. The terminal at the bottom shows the output of running the test file, which is a list of test results and a summary.

```
1 import unittest
2 from task_0 import calculator
3
4 class TestCalculator(unittest.TestCase):
5     def test_add(self):
6         self.assertEqual(calculator(10, 5, 'add'), 15)
7         self.assertEqual(calculator(-2, 3, 'add'), 1)
8
9     def test_subtract(self):
10        self.assertEqual(calculator(10, 5, 'subtract'), 5)
11        self.assertEqual(calculator(5, 10, 'subtract'), -5)
12
13    def test_multiply(self):
14        self.assertEqual(calculator(10, 5, 'multiply'), 50)
15        self.assertEqual(calculator(-2, 3, 'multiply'), -6)
16
17    def test_divide(self):
18        self.assertEqual(calculator(10, 5, 'divide'), 2.0)
19        self.assertEqual(calculator(9, 3, 'divide'), 3.0)
20
21    def test_divide_by_zero(self):
22        self.assertEqual(calculator(10, 0, 'divide'), "Error: Division by zero")
23
24    def test_invalid_operation(self):
25        self.assertEqual(calculator(10, 5, 'modulus'), "Error: Invalid operation")
26        self.assertEqual(calculator(10, 5, 'power'), "Error: Invalid operation")
27
28 if __name__ == '__main__':
29     unittest.main()
```

```
/test_task_0.py
15
5
50
2.0
Error: Division by zero
Error: Invalid operation
.....
Ran 6 tests in 0.000s

OK
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8>
```

Task Description#1

Use AI to generate test cases for `is_valid_email(email)` and then implement the validator function.

Requirements:

- Must contain @ and . characters.
- Must not start or end with special characters.
- Should not allow multiple @.

Expected Output#1

- Email validation logic passing all test cases

TASK 1:

The screenshot shows the Visual Studio Code interface with a Python file named `TASK1.py` open. The code defines a function `is_valid_email(email)` that checks if an email is valid based on several criteria: it must contain exactly one '@', have a dot in the local part, match a specific regex pattern, and have a valid domain structure. The script prompts the user to enter an email address. The terminal output shows the command being run and the successful validation of the email `2403a51021@sru.edu.in`.

```
1 import re
2 def is_valid_email(email):
3     if email.count('@') != 1:
4         return False
5     if '.' not in email:
6         return False
7     if not re.match(r'^[A-Za-z0-9][A-Za-z0-9@_-]*[A-Za-z0-9]$', email):
8         return False
9     if email.startswith('@') or email.endswith('@'):
10        return False
11    if email.startswith('.') or email.endswith('.'):
12        return False
13    return True
14
15 email = input("Enter your email: ")
16 if is_valid_email(email):
17     print("Valid email.")
18 else:
19     print("Invalid email.")
```

Terminal Output:

```
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8> & C:/Users/Rakesh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Rakesh/OneDrive/Desktop/Attachments/AIAC/Lab 8/TASK1.py"
Enter your email: 2403a51021@sru.edu.in
Valid email.
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8>
```

TEST CASE:

The screenshot shows the Visual Studio Code interface with a Python file named `test_TASK1.py` open. The code uses `unittest` to create a test class `TestIsValidEmail` with several test methods: `test_valid_emails` (valid emails), `test_missing_at_symbol` (missing '@'), `test_missing_dot` (missing dot), `test_invalid_start_end` (invalid start/end characters), `test_invalid_characters` (invalid characters), and `test_empty_string` (empty string). The terminal output shows the command being run and the successful execution of the tests.

```
1 import unittest
2 from TASK1 import is_valid_email
3
4 class TestIsValidEmail(unittest.TestCase):
5     def test_valid_emails(self):
6         self.assertTrue(is_valid_email("user@example.com"))
7         self.assertTrue(is_valid_email("john.doe123@domain.co"))
8         self.assertTrue(is_valid_email("a_b-c@sub.domain.com"))
9
10    def test_missing_at_symbol(self):
11        self.assertFalse(is_valid_email("userexample.com"))
12        self.assertFalse(is_valid_email("user@example.com"))
13
14    def test_missing_dot(self):
15        self.assertFalse(is_valid_email("user@examplecom"))
16        self.assertFalse(is_valid_email("user@com"))
17
18    def test_invalid_start_end(self):
19        self.assertFalse(is_valid_email("@user@example.com"))
20        self.assertFalse(is_valid_email("user@example.com@"))
21        self.assertFalse(is_valid_email(".user@example.com"))
22        self.assertFalse(is_valid_email("user@example.com."))
23
24    def test_invalid_characters(self):
25        self.assertFalse(is_valid_email("user!@example.com"))
26        self.assertFalse(is_valid_email("user#name@example.com"))
27        self.assertFalse(is_valid_email("user name@example.com"))
28
29    def test_empty_string(self):
30        self.assertFalse(is_valid_email(""))
31
32 if __name__ == "__main__":
33     unittest.main()
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8> & 'c:\Users\Rakesh\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Rakesh\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundle\libs\debugpy\launcher' '60612' '--' 'C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8\test_TASK1.py'
Enter your email: 2403a51021@sru.edu.in
Valid email.
.....
-----
Ran 6 tests in 0.002s

OK
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8> |
```

Task Description#2 (Loops)

- Ask AI to generate test cases for assign_grade(score) function. Handle boundary and invalid inputs.

Requirements

- AI should generate test cases for assign_grade(score) where: 90-100: A, 80-89: B, 70-79: C, 60-69: D, <60: F
- Include boundary values and invalid inputs (e.g., -5, 105, "eighty").

Expected Output#2

Grade assignment function passing test suite

TASK 2:

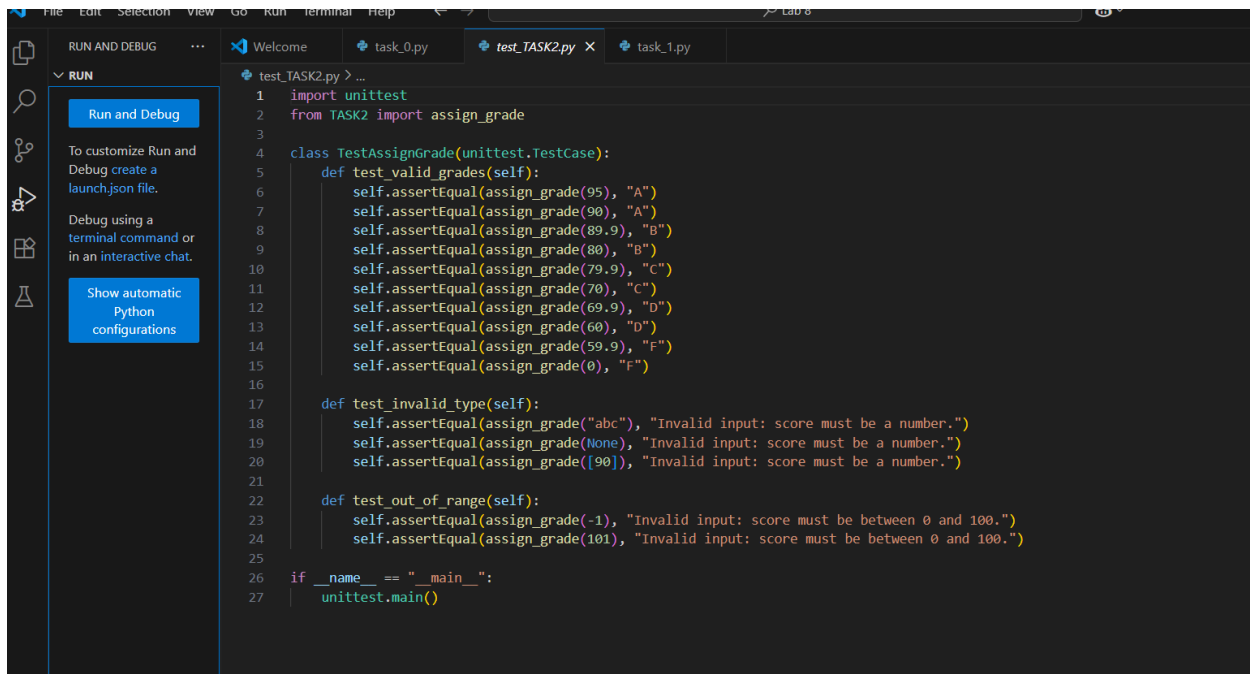
```
def assign_grade(score):
    if not isinstance(score, (int, float)):
        return "Invalid input: score must be a number."
    if score < 0 or score > 100:
        return "Invalid input: score must be between 0 and 100."
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

user_input = input("Enter the score: ")
try:
    score = float(user_input)
except ValueError:
    print("Invalid input: score must be a number.")
else:
    print(assign_grade(score))
```

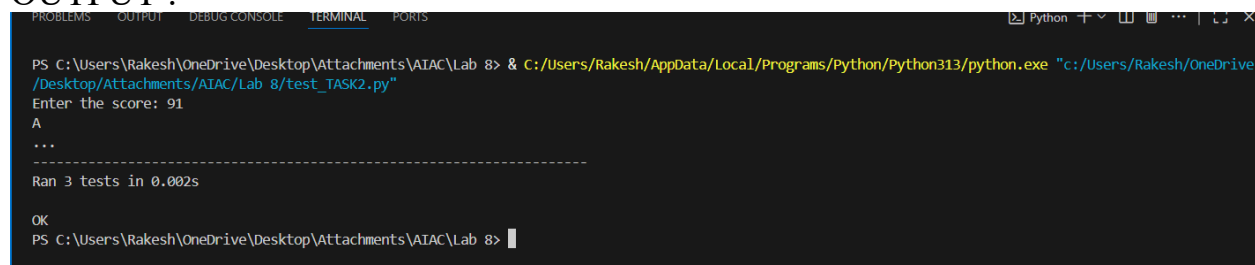
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/Desktop/Attachments/AIAC/Lab 8/TASK2.py
Enter the score: 90
A
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8> & C:/Users/Rakesh/AppData/Local/Programs/Python/Python313/python.exe "C:/Users/Rakesh/OneDrive/Desktop/Attachments/AIAC/Lab 8/TASK2.py"
Enter the score: 71
C
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8> |
```

TEST CASE:



OUTPUT :



Task Description#3

- Generate test cases using AI for is_sentence_palindrome(sentence). Ignore case, punctuation, and spaces

Requirement

- Ask AI to create test cases for is_sentence_palindrome(sentence) (ignores case, spaces, and punctuation).
- Example:
"A man a plan a canal Panama" → True

Expected Output#3

- Function returns True/False for cleaned sentences
- Implement the function to pass AI-generated tests.

TASK 3:

The screenshot shows the Visual Studio Code editor with a Python file named `TASK3.py`. The code implements a function `is_sentence_palindrome(sentence)` that checks if a sentence is a palindrome by removing punctuation, spaces, and converting to lowercase. The function uses `string` module for punctuation removal and `ch.isalnum()` for alphanumeric checking. A main block prompts the user to enter a sentence and prints the result.

```
1 import string
2
3 def is_sentence_palindrome(sentence):
4     # Remove punctuation, spaces, and convert to lowercase
5     cleaned = ''.join(
6         ch.lower() for ch in sentence if ch.isalnum()
7     )
8     return cleaned == cleaned[::-1]
9
10 if __name__ == "__main__":
11     print("Check if your own sentence is a palindrome (ignoring case, spaces, and punctuation):")
12     user_input = input("Enter a sentence: ")
13     if is_sentence_palindrome(user_input):
14         print("It's a palindrome!")
15     else:
16         print("It's not a palindrome.")
```

The terminal output shows the execution of the script, where the user enters "A man a plan a canal Panam" and the program correctly identifies it as not a palindrome.

TEST CASE:

The screenshot shows the Visual Studio Code editor with a Python file named `test_TASK3.py`. The code uses `unittest` to create a test suite for the `is_sentence_palindrome` function. It includes tests for simple palindromes, sentence palindromes, non-palindromes, empty strings, spaces and punctuation, case insensitivity, and numbers.

```
1 import unittest
2 from TASK3 import is_sentence_palindrome
3
4 class TestIsSentencePalindrome(unittest.TestCase):
5     def test_simple_palindrome(self):
6         self.assertTrue(is_sentence_palindrome("Madam"))
7
8     def test_sentence_palindrome(self):
9         self.assertTrue(is_sentence_palindrome("A man, a plan, a canal: Panama"))
10
11     def test_not_palindrome(self):
12         self.assertFalse(is_sentence_palindrome("Hello, world!"))
13
14     def test_empty_string(self):
15         self.assertTrue(is_sentence_palindrome(""))
16
17     def test_spaces_and_punctuation(self):
18         self.assertTrue(is_sentence_palindrome("Was it a car or a cat I saw?"))
19
20     def test_case_insensitivity(self):
21         self.assertTrue(is_sentence_palindrome("No lemon, no melon"))
22
23     def test_numbers(self):
24         self.assertTrue(is_sentence_palindrome("12321"))
25         self.assertFalse(is_sentence_palindrome("12345"))
26
27 if __name__ == "__main__":
28     unittest.main()
```

OUTPUT:

The screenshot shows the terminal output of running the unit tests. It displays the command to run the test file, followed by the execution of 7 tests in 0.001s, all of which passed (OK).

```
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8> & c:/Users/Rakesh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Rakesh/OneDrive/Desktop/Attachments/AIAC/Lab 8/test_TASK3.py"
.....
Ran 7 tests in 0.001s

OK
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8>
```

Task Description#4

- Let AI fix it Prompt AI to generate test cases for a ShoppingCart class (`add_item`, `remove_item`, `total_cost`).

Methods:

Add_item(name, price)

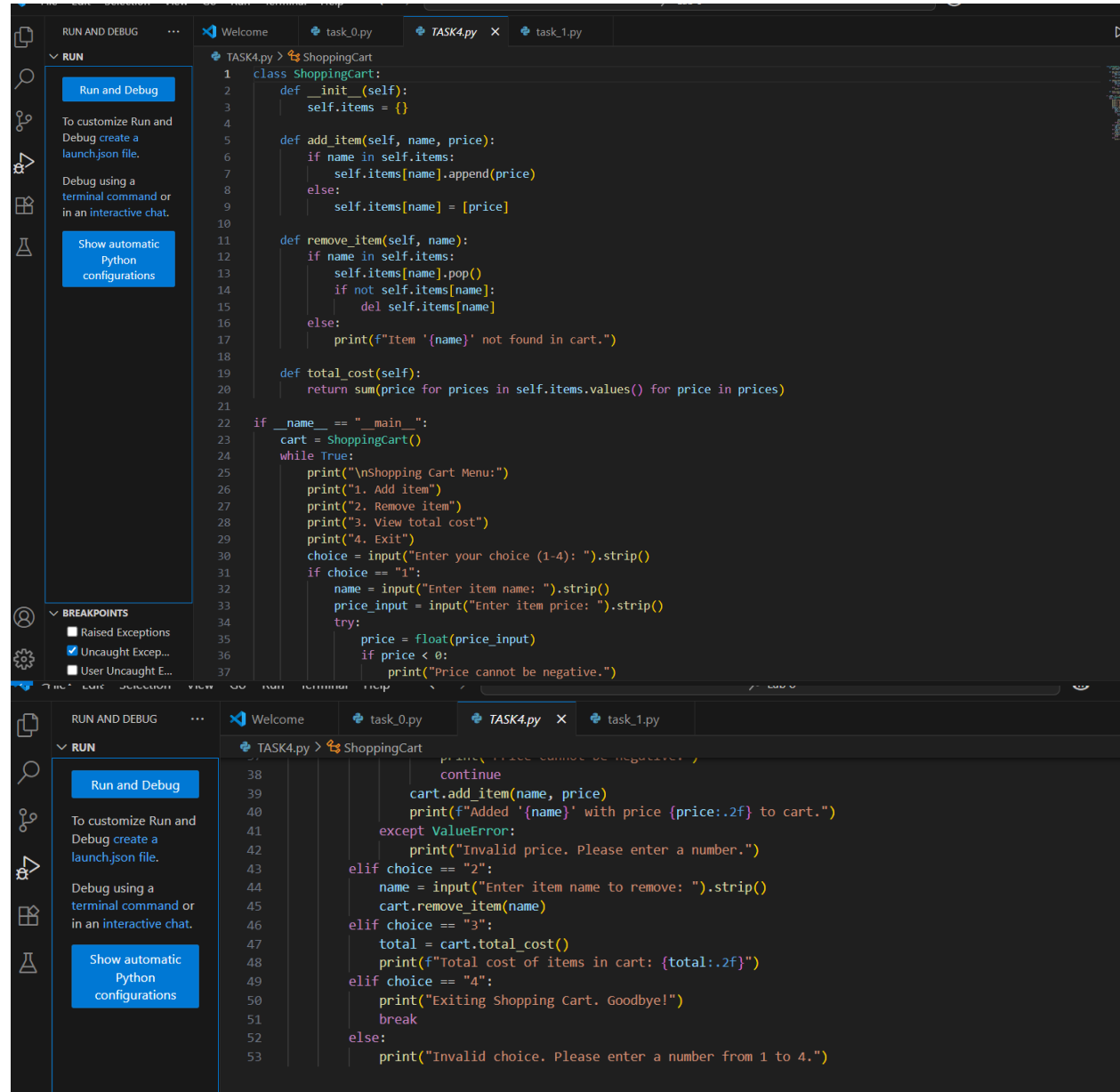
Remove_item(name)

Total_cost()

Expected Output#4

- Full class with tested functionalities

TASK 4:



```
1 class ShoppingCart:
2     def __init__(self):
3         self.items = {}
4
5     def add_item(self, name, price):
6         if name in self.items:
7             self.items[name].append(price)
8         else:
9             self.items[name] = [price]
10
11     def remove_item(self, name):
12         if name in self.items:
13             self.items[name].pop()
14             if not self.items[name]:
15                 del self.items[name]
16         else:
17             print(f"Item '{name}' not found in cart.")
18
19     def total_cost(self):
20         return sum(price for prices in self.items.values() for price in prices)
21
22 if __name__ == "__main__":
23     cart = ShoppingCart()
24     while True:
25         print("\nShopping Cart Menu:")
26         print("1. Add item")
27         print("2. Remove item")
28         print("3. View total cost")
29         print("4. Exit")
30         choice = input("Enter your choice (1-4): ").strip()
31         if choice == "1":
32             name = input("Enter item name: ").strip()
33             price_input = input("Enter item price: ").strip()
34             try:
35                 price = float(price_input)
36                 if price < 0:
37                     print("Price cannot be negative.")
38                     continue
39                 cart.add_item(name, price)
40                 print(f"Added '{name}' with price {price:.2f} to cart.")
41             except ValueError:
42                 print("Invalid price. Please enter a number.")
43         elif choice == "2":
44             name = input("Enter item name to remove: ").strip()
45             cart.remove_item(name)
46         elif choice == "3":
47             total = cart.total_cost()
48             print(f"Total cost of items in cart: {total:.2f}")
49         elif choice == "4":
50             print("Exiting Shopping Cart. Goodbye!")
51             break
52         else:
53             print("Invalid choice. Please enter a number from 1 to 4.")
```

OUTPUT:

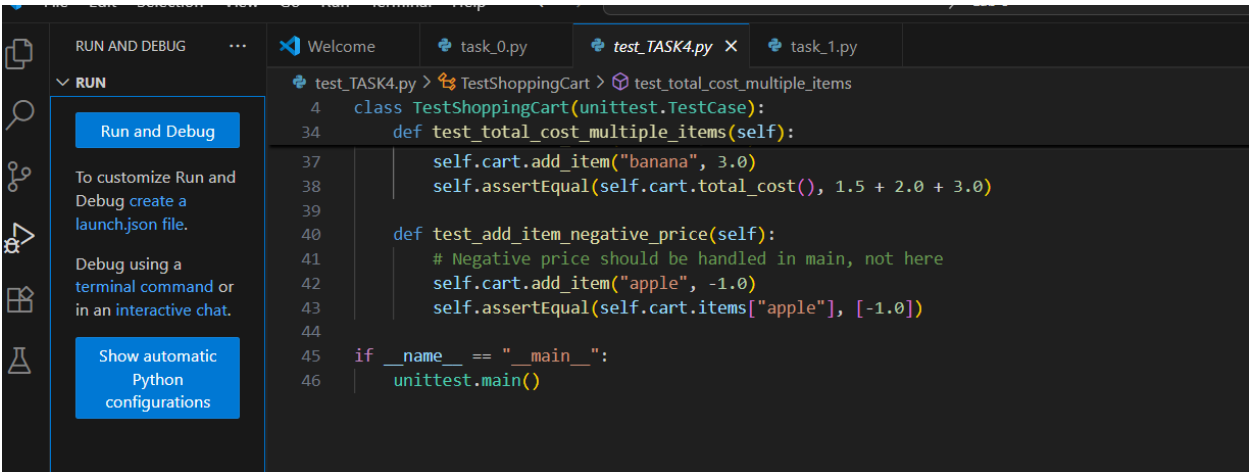
```
Python + v [ ] [ ] ... | [ ]
blms (Ctrl+Shift+M) kesh\OneDrive\Desktop\Attachments\AIAC\Lab 8> & C:/Users/Rakesh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Rakesh/OneDrive/Desktop/Attachments/AIAC/Lab 8/TASK4.py"

Shopping Cart Menu:
1. Add item
2. Remove item
3. View total cost
4. Exit
Enter your choice (1-4): 1
Enter item name: nuts
Enter item price: 190
Added 'nuts' with price 190.00 to cart.

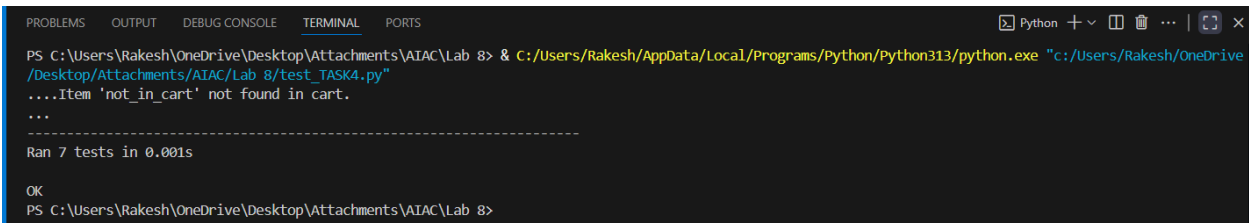
Shopping Cart Menu:
1. Add item
2. Remove item
3. View total cost
4. Exit
Enter your choice (1-4): 3
Total cost of items in cart: 190.00

Shopping Cart Menu:
1. Add item
2. Remove item
3. View total cost
4. Exit
Enter your choice (1-4): 2
Enter item name to remove: nuts

Shopping Cart Menu:
1. Add item
2. Remove item
3. View total cost
4. Exit
Enter your choice (1-4): 4
Exiting Shopping Cart. Goodbye!
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8>
```



OUTPUT:



Task Description#5

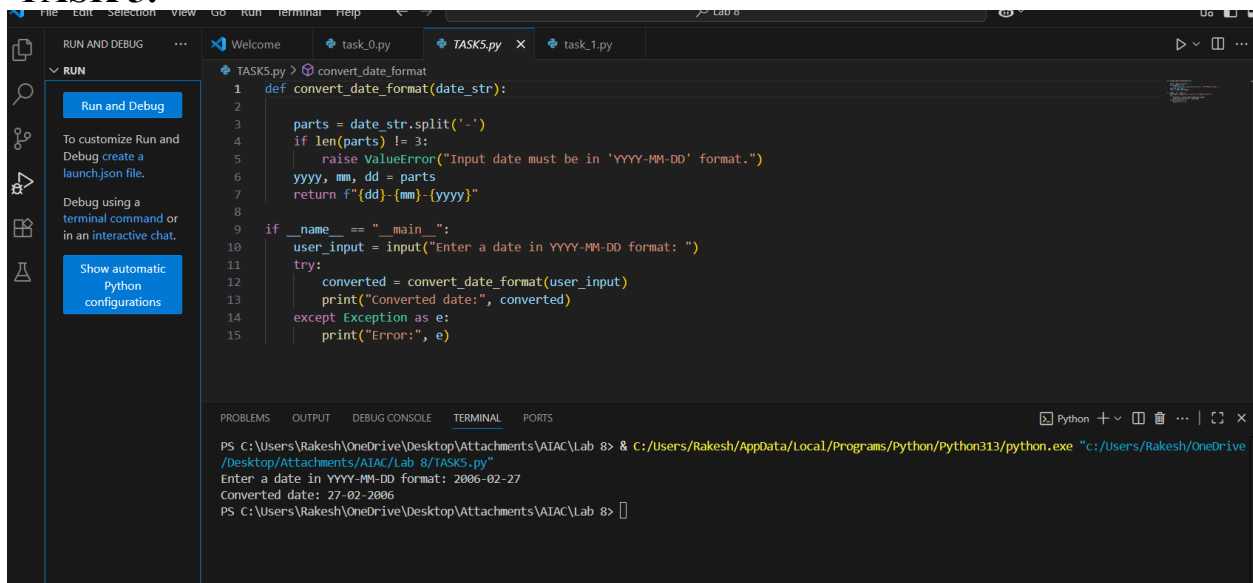
- Use AI to write test cases for convert_date_format(date_str) to switch from "YYYY-MM-DD" to "DD-MM-YYYY".

Example: "2023-10-15" → "15-10-2023"

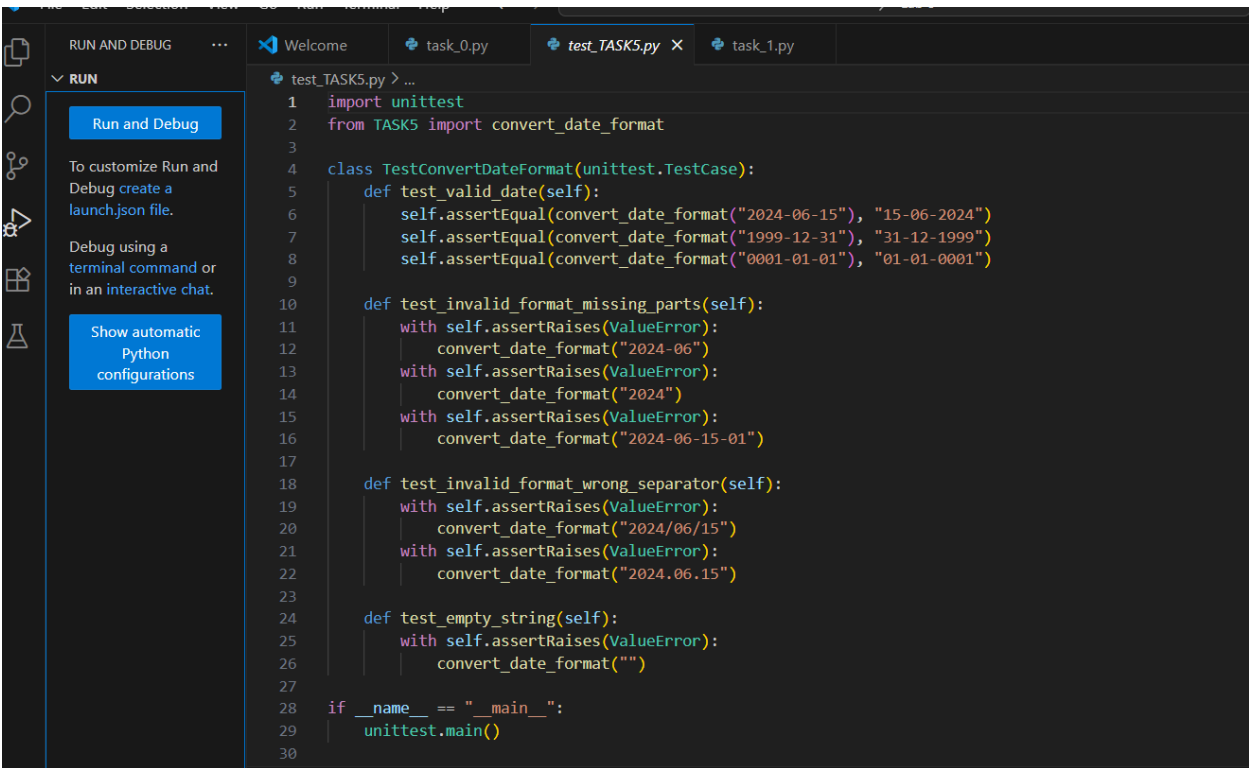
Expected Output#5

- Function converts input format correctly for all test cases

TASK 5:



TEST CASE:

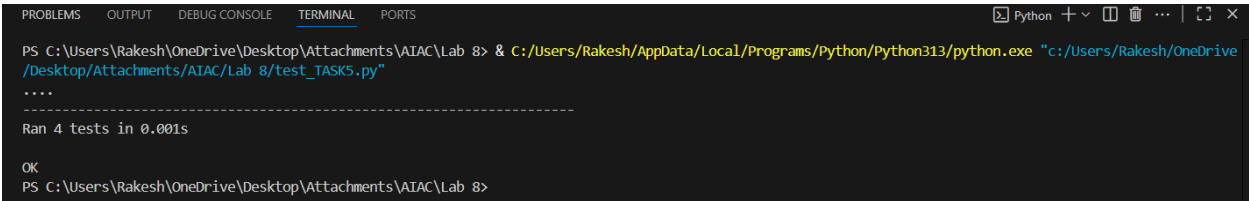


The screenshot shows the Visual Studio Code interface. The editor window displays a file named `test_TASK5.py` with the following Python code:

```
1 import unittest
2 from TASK5 import convert_date_format
3
4 class TestConvertDateFormat(unittest.TestCase):
5     def test_valid_date(self):
6         self.assertEqual(convert_date_format("2024-06-15"), "15-06-2024")
7         self.assertEqual(convert_date_format("1999-12-31"), "31-12-1999")
8         self.assertEqual(convert_date_format("0001-01-01"), "01-01-0001")
9
10    def test_invalid_format_missing_parts(self):
11        with self.assertRaises(ValueError):
12            convert_date_format("2024-06")
13        with self.assertRaises(ValueError):
14            convert_date_format("2024")
15        with self.assertRaises(ValueError):
16            convert_date_format("2024-06-15-01")
17
18    def test_invalid_format_wrong_separator(self):
19        with self.assertRaises(ValueError):
20            convert_date_format("2024/06/15")
21        with self.assertRaises(ValueError):
22            convert_date_format("2024.06.15")
23
24    def test_empty_string(self):
25        with self.assertRaises(ValueError):
26            convert_date_format("")
27
28 if __name__ == "__main__":
29     unittest.main()
30
```

The left sidebar shows the 'RUN AND DEBUG' section with a 'Run and Debug' button and instructions: 'To customize Run and Debug create a launch.json file.' and 'Debug using a terminal command or in an interactive chat.' There is also a 'Show automatic Python configurations' button.

OUTPUT:



The screenshot shows the VS Code terminal window with the following output:

```
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8> & C:/Users/Rakesh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Rakesh/OneDrive/Desktop/Attachments/AIAC/Lab 8/test_TASK5.py"
....
-----
Ran 4 tests in 0.001s

OK
PS C:\Users\Rakesh\OneDrive\Desktop\Attachments\AIAC\Lab 8>
```